

S - Single Responsibility

Definition

One class should have only one reason to change.

□ Before

```
class OrderService {  
    saveOrder();  
    sendEmail();  
    printInvoice();  
}
```

□ After

```
OrderService, EmailService,  
InvoiceService
```

Problem: Many responsibilities in one class.

Advantages: Easy maintenance Low coupling

Interview Tip: One class = One responsibility

Problem

مسئول عن OrderService:

- Saving Orders
- Sending Emails
- Printing Invoices

OrderService. لو طريقة إرسال الإيميل اتغيرت هنعديل

OrderService. ولو الفاتورة اتغيرت هنعديل

Reason to Change. يعني عنده أكثر من

Advantages

✓ Easy Maintenance

✓ Easy Testing

✓ Better Reusability

✓ Low Coupling

O - Open/Closed

Definition

Open for extension, closed for modification.

□ Before

```
if (type=='Card') ...  
else if(...)
```

□ After

```
interface Payment  
class CardPayment
```

Problem: Modify old code for every payment.

Advantages: Extensible

Interview Tip: Extend don't modify

PaymentService جديدة لازم تعدل كل مرة تضيف

OCP. وده بيكسر

Advantages:

- ✓ Easy Extension
- ✓ No modification of existing code
- ✓ Less Bugs
- ✓ Better Scalability

L - Liskov

Definition

Child replaces parent safely.

□ Before

```
Bird.fly(); Penguin extends Bird
```

□ After

```
Bird, FlyingBird, Penguin
```

Problem: Penguin can't fly.

Advantages: Safe polymorphism

Interview Tip: Child must honour parent contract

Penguin is a Bird

لكن لا يستطيع الطيران.

يكسر البرنامج Penguin بـ Bird إذن استبدال

Advantages

✓ Correct Inheritance

✓ No Unexpected Errors

✓ Better Polymorphism

I - Interface Segregation

Definition

Don't force unused methods.

□ Before

Worker (work, eat, sleep)

□ After

Workable/Eatable/Sleepable

Problem: Robot forced to eat.

Advantages: Small interfaces

Interview Tip: Keep interfaces focused

Robot لا يأكل ولا ينام.

Methods. ومع ذلك أُجبر على تنفيذ هذه الـ

Advantages

✓ Small Interfaces

✓ Flexible Design

✓ Easier Maintenance

D - Dependency Inversion

Definition
Depend on abstractions.

□ Before
`new MySQLDatabase()`

□ After
`Database interface + DI`

Problem: Tight coupling.

Advantages: Loose coupling

Interview Tip: Depend on interfaces

OrderService مرتبط مباشرة بـ MySQLDatabase.

OrderService ستعدل PostgreSQL إذا استخدمت

Advantages

- ✓ Loose Coupling
- ✓ Easy Testing
- ✓ Easy Database Switching
- ✓ Used heavily in Spring Dependency Injection

SOLID Cheat Sheet

S: One Responsibility
O: Extend, don't modify
L: Child replaces parent safely
I: Small focused interfaces
D: Depend on interfaces.

Principle	Meaning	Remember
S	Single Responsibility	One Class → One Responsibility
O	Open/Closed	Extend without modifying existing code
L	Liskov Substitution	Child should safely replace Parent
I	Interface Segregation	Small, focused interfaces
D	Dependency Inversion	Depend on Interfaces, not Concrete Classes